

Вычисление аналитической метрики на большом графе

Out-of-core LeaderRank с параллельной обработкой

7 июня 2026 г.

Аннотация

В работе вычисляется метрика центральности **LeaderRank** на ориентированном невзвешенном графе, который не помещается в оперативную память. Рёбра хранятся на диске и читаются потоком, а в памяти остаются только массивы размером с число вершин. Счёт идёт во много потоков и не зависит от их числа: результат получается один и тот же бит в бит. В отчёте разобраны выбор метрики, варианты хранения графа, устройство решения, анализ сложности и точности, а также замеры на синтетических и реальных графах.

1 Введение

1.1 Задача

Нужно посчитать аналитическую метрику на ориентированном невзвешенном графе. Главная сложность в том, что граф целиком не влезает в оперативную память. В условии приведён пример: памяти 128 МБ, а на граф нужно около 1 ГБ. Значит, держать весь граф в ОЗУ нельзя, надо работать с диском.

Условие добавляет ещё несколько требований.

- **Обработка с диска (out-of-core).** В памяти можно держать только данные порядка числа вершин, но не порядка числа рёбер.
- **Гипер-узлы.** Встречаются вершины со степенью на порядки выше средней (средняя степень 50, а у одной вершины 50 000). Решение не должно ломаться на таких вершинах.
- **Параллелизм.** Узел один, но с несколькими ядрами. Нужно занять все ядра.
- **Достоверность.** Результат не должен зависеть от случайности и должен повторяться от запуска к запуску.
- **Разумное время.** На больших графах счёт должен укладываться в приемлемое время.

Формат ввода и вывода. Он зафиксирован условием. Вход — CSV с заголовком, первые два столбца `from`, `to`, идентификаторы вершин типа `int32`, ребро направлено `from` → `to`, граф невзвешенный. Выход — CSV со столбцами `vertex`, `rank`, по одному числу на вершину.

1.2 Что сделано

Выбрана метрика **LeaderRank** — вариант PageRank без настраиваемых параметров. Решение написано на Java (OpenJDK 21). Рёбра графа лежат на диске и читаются потоком, в памяти живут только массивы размера $\mathcal{O}(n)$. Счёт распараллелен по ядрам и даёт один и тот же результат при любом числе потоков. Лимит памяти задаётся снаружи (через `-Xmx` и `cgroup`), а программа сама подстраивается под него. Подробные инструкции по сборке и запуску вынесены в **README**, здесь же описано устройство решения.

1.3 Как читать отчёт

Раздел 2 — про метрику LeaderRank и почему выбрана именно она. Раздел 3 — варианты хранения графа и выбор одного из них. Разделы 4–6 — устройство решения: архитектура, параллелизм, память. Раздел 7 — анализ сложности и точности. Раздел 8 — тесты и замеры. Раздел 9 — выводы.

2 Метрика LeaderRank

2.1 Что это и как работает

LeaderRank — метрика центральности, то есть она даёт каждой вершине одно число «важности». Это модификация PageRank, придуманная для поиска лидеров в социальных сетях. Главное отличие от PageRank в том, что у неё нет настраиваемых параметров.

Идея простая. В граф добавляют одну искусственную вершину — *фоновый узел* g (ground node). Его соединяют двусторонними рёбрами со всеми остальными вершинами ($g \rightarrow i$ и $i \rightarrow g$ для каждой вершины i). По такому расширенному графу из $n + 1$ вершины запускают обычное случайное блуждание, без телепортации и без коэффициента затухания.

Почему это работает. Фоновый узел связан со всеми в обе стороны, поэтому расширенный граф сильно связный и апериодичный. Из теории цепей Маркова отсюда следует, что у блуждания есть единственное стационарное распределение, а степенной метод к нему сходится. И всё это без подбора параметров. Заодно фоновый узел играет ту же роль, что телепортация в PageRank, но встроен прямо в структуру графа.

2.2 Формула

Пусть у вершины i исходящая степень $\deg^{\text{out}}(i)$. В расширенном графе добавляется ребро к фоновому узлу, поэтому исходящая степень становится $\deg^{\text{out}}(i) + 1$. На каждом шаге каждая вершина i отдаёт по $s_i / (\deg^{\text{out}}(i) + 1)$ каждому реальному соседу и столько же фоновому узлу, а фоновый узел отдаёт по s_g / n каждой из n вершин. Начальное состояние: $s_i = 1$ у всех реальных вершин, $s_g = 0$. Суммарная «масса» равна n и сохраняется на всех шагах. После сходимости масса фонового узла поровну возвращается вершинам: $s_i \leftarrow s_i + s_g / n$.

Удобно ввести вклад вершины, который считается раз за итерацию:

$$\text{contrib}[j] = \frac{s_{\text{old}}[j]}{\deg^{\text{out}}(j) + 1}.$$

Тогда обновление состояния записывается так:

$$\text{base} = s_{\text{old}}[g]/n, \quad (1)$$

$$s_{\text{new}}[i] = \text{base} + \sum_{j:j \rightarrow i} \text{contrib}[j], \quad i = 1, \dots, n, \quad (2)$$

$$s_{\text{new}}[g] = \sum_{j=1}^n \text{contrib}[j]. \quad (3)$$

Формулы (1) и (3) — дешёвые операции порядка $\mathcal{O}(n)$. Дорогая часть одна — сумма (2) по рёбрам для каждого приёмника. В матричном виде это $s_{\text{new}} = \text{base} \cdot \mathbf{1} + A^\top \cdot \text{contrib}$, то есть умножение транспонированной матрицы смежности на вектор. Важно, что $2n$ рёбер фонового узла физически не хранятся: его вклад считается двумя формулами.

2.3 Итерация — это один проход по рёбрам

Ключевой факт: умножение разреженной матрицы на вектор — это один проход по списку рёбер. Чтобы посчитать $A^\top \cdot \text{contrib}$, достаточно для каждого ребра $j \rightarrow i$ прибавить $\text{contrib}[j]$ к ячейке i . Матрицу в памяти держать не нужно, нужен только поток рёбер. Именно на этом держится всё решение: рёбра можно читать с диска по очереди, а в памяти хранить лишь векторы размера $\mathcal{O}(n)$.

Останавливаемся, когда состояние почти перестаёт меняться. Критерий — норма L_1 разности $\sum_i |s_{\text{new}}[i] - s_{\text{old}}[i]| < \varepsilon$. Про конкретный порог и число итераций — в разделе 7. Масса сохраняется, поэтому нормировать вектор на каждом шаге не нужно. Редкая перенормировка к сумме n применяется только чтобы не накапливалась ошибка округления.

2.4 Где это полезно

LeaderRank ищет влиятельные узлы. Исходно его применяли к социальной сети Delicious для поиска лидеров мнений. По сути он годится везде, где применим PageRank: ранжирование страниц, важность пользователей, графы цитирования, рекомендации. Его берут, когда не хочется подбирать коэффициент затухания или когда в графе много висячих вершин.

2.5 Почему выбрана именно она

Условие разрешает любую метрику, применяемую ко всему графу, и рекомендует PageRank, LeaderRank, Katz, Jaccard, Dice. LeaderRank выбран по нескольким причинам.

1. **Подходит формат.** LeaderRank даёт одно число на вершину, что прямо ложится в вывод `vertex, rank`. Пример выходного файла в условии так и называется — `leader_rank.csv`.

2. **Естественно считается с диска.** Каждая итерация — один последовательный проход по рёбрам. Значит рёбра (это и есть тот «гигабайт») читаются потоком с диска, а в памяти остаются только векторы $\mathcal{O}(n)$. Память ограничена числом вершин, а не числом рёбер.
3. **Не боится гипер-узлов.** Списки смежности не материализуются в памяти. Гипер-узел степени 50 000 — это просто 50 000 записей в потоке рёбер на диске. Вклад вершины делится на её степень, никакого взрыва нет.
4. **Легко параллелится.** Проход по рёбрам просто делится между потоками.
5. **Детерминирован.** Случайности в алгоритме нет. При фиксированном порядке сложения результат повторяется.
6. **Выигрывает у соседей по семейству.** В отличие от PageRank, не нужен параметр d и не нужна отдельная обработка висячих вершин — её берёт на себя фоновый узел. В отличие от Katz, не нужен спектральный радиус. Меры Jaccard и Dice вообще не подходят: требуют пересечения списков смежности (случайный доступ — худший случай для диска) и катастрофически взрываются на гипер-узлах (узел степени D участвует в $\mathcal{O}(D^2)$ парах). При этом схема вычислений у LeaderRank та же, что у PageRank, поэтому при желании PageRank поддерживается почти тем же кодом.

3 Способы представления графа

3.1 Рамки решения

Прежде чем выбирать представление, стоит зафиксировать рамки, в которых решается задача. Рёбер $\mathcal{O}(m)$ — это и есть «гигабайт» из условия, они не обязаны помещаться в память и живут на диске. А структуры размером с число вершин, $\mathcal{O}(n)$ (векторы значений, степени, указатели), наоборот, обязаны помещаться в ОЗУ: без них повершинную метрику не посчитать. Граница такая — $\mathcal{O}(m)$ не ограничено, $\mathcal{O}(n)$ обязано влезать.

Отсюда сразу отпадают подходы, которые выгружают на диск даже сами векторы значений $\mathcal{O}(n)$, например Parallel Sliding Windows из GraphChi. Для этой задачи они избыточны: при «гигабайте» рёбер вершин порядка 2–3 млн, а их векторы — это лишь десятки МБ, и они спокойно помещаются в память. Случай, когда не влезает даже $\mathcal{O}(n)$, — это уже другая задача, и она вне рамок данной работы.

3.2 Push или pull

Ядро метрики — сумма по рёбрам (2). Организовать этот проход можно двумя способами, и именно способ обработки определяет, как выгодно хранить граф.

Push (по источнику). Обход идёт по рёбрам, сгруппированным по источнику j , и $\text{contrib}[j]$ прибавляется в ячейку приёмника $s_{\text{new}}[i]$. Хранение здесь простое: рёбра можно держать как есть, в порядке поступления, сортировать ничего не надо. Но при параллельном проходе несколько потоков добавляют в одну и ту же ячейку популярного приёмника одновременно — возникает гонка. Пришлось бы ставить

блокировки или атомарные операции, а порядок атомарных сложений не определён, поэтому результат менялся бы в младших разрядах от запуска к запуску.

Pull (по приёмнику). Обход идёт по приёмникам i , и для каждого собирается сумма $s_{\text{new}}[i] = \text{base} + \sum_{j \rightarrow i} \text{contrib}[j]$. У каждой ячейки результата ровно один писатель, гонок нет. Зато появляется цена на хранении: чтобы входящие рёбра приёмника i лежали подряд, рёбра нужно один раз сгруппировать и отсортировать по приёмнику. Это разовая работа на этапе предобработки.

3.3 Выбор

Выбран pull. Решающим стало требование воспроизводимости. Push не требует сортировки, но его гонки и неопределённый порядок сложения несовместимы с условием «результат не должен зависеть от случайности». Pull стоит разовой сортировки по приёмнику, зато на каждой из десятков итераций даёт проход без блокировок, последовательное чтение с диска и детерминированный результат. Разовая сортировка окупается уже на первых итерациях.

Из выбора pull прямо следует формат хранения: граф группируется по приёмнику и сортируется. Как он раскладывается на диске и как строится за несколько внешних проходов — в разделе 4.

4 Архитектура решения

4.1 Два этапа

Решение состоит из предобработки (один раз) и итераций (много раз).

- **Предобработка** переводит входной CSV в компактный бинарный вид, удобный для потокового чтения. Это разовая плата, которая окупается: бинарный файл меньше CSV и упорядочен так, что итерация сводится к последовательному чтению.
- **Итерация** — это шаг степенного метода LeaderRank (2). Бинарный файл читается потоком, считается ядро. В памяти живут только векторы $\mathcal{O}(n)$.

После сходимости плотные идентификаторы переводятся обратно в исходные `int32`, и результат пишется в выходной CSV.

4.2 Формат хранения

Ядру нужно для каждого приёмника i перечислить его входящих соседей j . То есть нужна транспонированная матрица A^T — граф, сгруппированный по приёмнику. Храним его как разреженный столбцовый формат (CSC), разделённый на две части по принципу «что влезает в ОЗУ — в ОЗУ, что нет — на диск».

- `sourcesPtr` — массив `int[n+1]` указателей. Входящие рёбра приёмника i лежат в файле от позиции `sourcesPtr[i]` до `sourcesPtr[i+1]`. Размер $\mathcal{O}(n)$, в ОЗУ.
- `sources` — поток источников j , сгруппированных по приёмнику. Размер $\mathcal{O}(m)$, это и есть «гигабайт», лежит на диске и читается потоком. Каждое ребро — это 4-байтовое целое (little-endian), только источник. Приёмник восстанавливается по положению в файле через `sourcesPtr`, поэтому повторно он не хранится.

- **outdeg** — массив `int[n]` исходящих степеней (нужен для знаменателя $\deg^{\text{out}}(j)+1$). В ОЗУ.
- **originalIds** — соответствие «плотный `id` → исходный `int32`» для вывода. В ОЗУ, нужен только при записи результата.

4.3 Конвейер предобработки

Предобработка — это по сути внешняя сортировка: рёбра перегруппировываются по приёмнику, не загружая их все в память. Именно эту «работу с памятью» условие требует написать самим. Этапы:

1. **Один проход по CSV: переименование `id` и подсчёт степеней.** Исходные `int32` разрежены, поэтому они переводятся в плотный $0..n-1$ хеш-таблицей «`int`→`int`» на примитивах. Таблица размера $\mathcal{O}(n)$ (вершины влезают в память, в отличие от рёбер). Заодно считаются входящие степени (для `sourcesPtr`) и исходящие (для знаменателя). Порядок присвоения плотных `id` фиксирован, по первому появлению. Это часть детерминизма. Чтобы не разбирать CSV второй раз, на этом же проходе рёбра в плотных `id` пишутся во временный бинарный файл.
2. **Планирование корзин.** Диапазон приёмников делят на корзины так, чтобы в каждую попадало не больше M рёбер. Размер M подбирается под доступную память. Если у одной вершины входящая степень больше M , она не делится и попадает в отдельную «большую» корзину.
3. **Раздача рёбер по корзинам.** Временный бинарный файл читается, каждое ребро дописывается в файл своей корзины. Записи последовательные, буферы фиксированного размера. Если корзин больше, чем можно открыть файлов под лимитом памяти, раздача идёт волнами: источник пересчитывается несколько раз, по группе корзин за проход.
4. **Сортировка корзин и сборка файла `sources`.** Обычная корзина целиком загружается в память, сортируется по ключу «(приёмник, источник)» и дописывается в итоговый файл. Большая корзина (гипер-узел) сортируется внешней сортировкой слиянием: режется на куски по M , каждый кусок сортируется и сливается потоком. Так память на одну корзину остаётся $\mathcal{O}(M)$ при любой степени вершины.

Итог предобработки — файл `sources` на диске плюс массивы $\mathcal{O}(n)$ в памяти. Дальше итерация работает только с ними.

Почему детерминированно. Итоговая раскладка (по приёмнику, источники по возрастанию) — это чистая функция от графа. Она не зависит ни от M , ни от границ корзин, ни от числа потоков. Это проверяется тестом: $M = \infty$ и $M = \text{маленькое}$ дают байт в байт одинаковый файл.

5 Параллельная обработка

Pull выбран в разделе 3. Здесь — как этот проход распараллелить, не получив гонок и не потеряв детерминизм.

5.1 Без блокировок и детерминизм

Каждый поток берёт непрерывный диапазон приёмников и пишет только в свой непересекающийся кусок s_{new} . Массивы `contrib` и `sold` общие и только на чтение. Отсюда два следствия.

- **Горячий цикл без блокировок.** В проходе $\mathcal{O}(m)$ нет ни атомиков, ни `synchronized`. Единственная синхронизация — один барьер на итерацию.
- **Детерминизм, не зависящий от числа потоков.** Никакие два потока не пишут в одну ячейку, а порядок сложения внутри ячейки задан порядком рёбер в файле. Поэтому результат получается бит в бит одинаковым при 1 и при 24 потоках и от запуска к запуску. Планировщик потоков на ответ не влияет: ни одна ячейка не делится между потоками, а порядок суммирования в ней зафиксирован на диске.

5.2 Балансировка и гипер-узлы

Делить поровну по числу вершин нельзя. Из-за степенного распределения равные по числу вершин куски содержат очень разное число рёбер, и поток с гипер-узлом становится отстающим. Поэтому:

- границы кусков выбираются так, чтобы уравнивать число рёбер (по префиксной сумме входящих степеней).
- кусков создаётся больше, чем потоков (в несколько раз), и пул разбирает их по мере освобождения. Это сглаживает остаточный перекоп.
- гипер-узел в модели `pull` неделим (у ячейки один владелец), но в абсолютных числах он мал. Входящая степень 50 000 — это около 0,2 МБ источников, микросекунды работы.

5.3 Реализация на Java

- **Пул потоков и барьер.** Фиксированный `ExecutorService` на P потоков по числу ядер. На каждой итерации задачи-куски отдаются в `invokeAll`, который ждёт завершения всех. Это ожидание и есть барьер итерации: пока все приёмники не пересчитаны, следующая итерация не начнётся. Отдельный `CyclicBarrier` не нужен.
- **Корректность по модели памяти Java.** Потоки пишут в s_{new} без `volatile` и без блокировок. Это безопасно, потому что завершение задачи и его наблюдение через `invokeAll` задают отношение `happens-before`. Всё, что поток записал, видно главному потоку, когда тот увидел задачу завершённой. Значит записи итерации t опубликованы до чтений итерации $t+1$.
- **Разбиение диапазонами против false sharing.** Поток пишет непрерывный кусок, поэтому делиться с соседом может только одна кэш-линия на границе. При кусках в тысячи элементов это незаметно. Чередующееся разбиение, наоборот, посадило бы соседние потоки на общие кэш-линии.
- **Детерминированные суммы.** Глобальные суммы (масса фонового узла и норма сходимости) считаются в фиксированном порядке: каждый поток отдаёт частичную сумму, а главный поток складывает их по порядку номеров.

`DoubleAdder` и `stream().sum()` с неопределённым порядком сознательно не используются. Потоки платформенные, а не виртуальные: виртуальные хороши для блокирующего ввода-вывода, но не ускоряют счёт.

- **Мало мусора.** Массивы s_{old} , s_{new} , `contrib`, `outdeg`, `sourcesPtr` выделяются один раз. Между итерациями s_{old} и s_{new} просто меняются ссылками. Горячий цикл почти не создаёт мусора, поэтому Serial GC справляется даже при крошечной куче.

5.4 Ввод-вывод

Граф не помещается в память, поэтому файл `sources` перечитывается с диска на каждой итерации. Закэшировать его целиком нельзя, в этом суть задачи. Каждый поток читает свои куски последовательно, держа только буфер фиксированного размера (64 КиБ), а не весь кусок. Буфер свой на каждый поток и переиспользуется между итерациями, поэтому суммарная память на буферы — это $P \cdot B$, а не P копий чего-то большого. На SSD упреждающее чтение ОС неплохо совмещает ввод-вывод со счётом.

6 Память и соблюдение лимита

6.1 Принцип out-of-core

Данные делятся по размеру. Структуры $\mathcal{O}(n)$ (по числу вершин) живут в ОЗУ. Структуры $\mathcal{O}(m)$ (по числу рёбер) живут на диске и читаются потоком. Поскольку «гигабайт» — это именно рёбра, потребление памяти определяется числом вершин, а не рёбер. Отсюда следствие, заметное на практике: размер RSS зависит от числа вершин, а не от размера CSV-файла. Меньший по объёму файл с большим числом вершин требует больше памяти, чем больший файл с малым числом вершин.

6.2 Из чего складывается память

Потребление ОЗУ складывается из трёх частей.

- **Корзина А — общие векторы $\mathcal{O}(n)$.** Три `double[n]` (s_{old} , s_{new} , `contrib`) и несколько `int[n]` (`sourcesPtr`, `outdeg`, `originalIds`) — это примерно 36 байт на вершину. Для $n \approx 4,8$ млн — около 170 МБ. Эта часть общая для всех потоков и от их числа не зависит.
- **Корзина В — попоточные буферы.** Один буфер чтения на поток (64 КиБ). Всего $P \cdot B$ плюс стеки потоков — единицы МБ. Это единственная часть, растущая с числом потоков.
- **Корзина С — накладные расходы JVM.** Metaspace, кэш кода, структуры сборщика — около 20–30 МБ. Serial GC держит эту часть малой.

Основной объём — корзина А — общий и не уменьшается с ростом числа потоков. С потоками растёт только небольшая корзина В.

6.3 Лимит памяти и его проверка

Ограничение памяти задаётся снаружи, а программа подстраивает под него своё поведение. Доступный объём она узнаёт из максимального размера кучи (значение `-Xmx`, оно же видно как лимит `cgroup` в контейнере) и под него подбирает размер корзины M при предобработке. Один рычаг (`-Xmx` или `cgroup`) задаёт и реальный потолок памяти, и параметры предобработки.

Чтобы симулировать ограничение из условия и проверить, что оно действительно соблюдается, использованы два механизма.

- **Лимит кучи JVM (`-Xmx`).** Ограничивает кучу Java. Нужен, но его одного мало: вся память процесса — это куча плюс стеки, `metaspace`, кэш кода, буферы вне кучи и страничный кэш. Часть из этого лежит вне `-Xmx`.
- **Лимит всего процесса через `cgroup v2` (основной).** JVM запускается внутри ограничения `cgroup`, и ядро ОС ограничивает всё сразу, снимая процесс по ООМ при нарушении:

```
systemd-run --user --scope -p MemoryMax=256M -p MemorySwapMax=0 \  
  java -Xmx128m -XX:+UseSerialGC ...
```

Своп отключён (`MemorySwapMax=0`), иначе куча незаметно вытеснялась бы в своп и лимит стал бы фикцией.

Само завершение запуска под лимитом `cgroup` и есть доказательство: при превышении процесс был бы снят ядром. Дополнительно в конце запуска печатается пиковый RSS (значение `VmHWM` из `/proc/self/status`) — получается конкретное число вида «пик RSS = 168 МиБ при лимите 256 МиБ».

Деталь про `-Xmx`. Он ограничивает только кучу. Поверх неё JVM тратит ещё 60–80 МБ на нативную часть (сам `libjvm`, кэш кода, стеки). Поэтому, чтобы удержать RSS под лимитом C , `-Xmx` стоит брать примерно на 80 МБ меньше C , а не равным C .

6.4 Контрольный контрпример

Чтобы показать ценность `out-of-core`, в проект включён намеренно наивный вариант, который держит весь граф в памяти (вложенные списки в ОЗУ). Под тем же лимитом `cgroup` он падает, а потоковый движок завершается. Падает он двумя способами. При куче, равной потоковой, ему не хватает кучи Java (`OutOfMemoryError`). При щедрой куче (`-Xmx1g`) его снимает ядро по ООМ (сигнал KILL, код 137). Алгоритм и лимит те же, отличается только способ хранения.

6.5 Ограничения подхода

- В ОЗУ обязаны помещаться структуры $\mathcal{O}(n)$ — векторы значений и указатели (раздел 3.1). Если не влезают даже они, это уже другая задача (выгрузка самих векторов на диск, как в GraphChi PSW), и она вне рамок работы.

- На очень больших графах узкое место — скорость последовательного чтения с диска (файл `sources` перечитывается на каждой итерации). Компактный формат поднимает этот потолок, но саму природу ограничения не убирает.
- 128 МБ — это тесно для JVM. Нативный пол (≈ 70 МБ) плюс векторы $\mathcal{O}(n)$ при n около 2,7 млн (≈ 108 МБ) уже превышают 128 МБ. При таком жёстком лимите это аргумент в пользу C++. На Java тот же подход работает при чуть большем лимите.

7 Сложность и точность

Обозначения: n — число вершин, m — число рёбер, P — число потоков, T — число итераций до сходимости, B — размер попоточного буфера.

7.1 Время

Предобработка (один раз). Проход по входу с подсчётом степеней и переименованием — $\mathcal{O}(m)$. Префиксная сумма для `sourcesPtr` — $\mathcal{O}(n)$. Раздача рёбер по корзинам — $\mathcal{O}(m)$. Сортировка корзин — около $\mathcal{O}(m \log(m/K))$ при K корзинах, с малым множителем (сортируются короткие корзины). Итого предобработка — $\mathcal{O}(m \log m)$ по времени и $\Theta(m)$ по вводу-выводу (несколько последовательных проходов).

Одна итерация. Вычисление `contrib`, инициализация s_{new} , суммы — $\mathcal{O}(n)$. Ядро (2) — ровно один проход по файлу `sources`, $\mathcal{O}(m)$ сложений и $\Theta(m)$ последовательного чтения. Распараллеливание делит работу на P потоков. При сбалансированных по числу рёбер кусках счётное время итерации — $\mathcal{O}((n+m)/P)$, а время чтения — $\Theta(m/\beta)$, где β — эффективная скорость диска. Полное время:

$$T_{\text{итер}} = \mathcal{O}\left(T \cdot \frac{n+m}{P}\right) (\text{счёт}) + \Theta\left(T \cdot \frac{m}{\beta}\right) (\text{ввод-вывод}).$$

Реальное время — максимум из двух слагаемых. На больших графах доминирует ввод-вывод (T перечитываний `sources`), поэтому число читаемых байт сведено к минимуму, а чтение идёт последовательно.

7.2 Память

ОЗУ: $\Theta(n)$ на общие векторы + $\mathcal{O}(P \cdot B)$ на буферы + $\mathcal{O}(1)$ накладных JVM. Главное — не зависит от m и от наличия гипер-узлов. Диск: $\Theta(m)$ на `sources` + $\Theta(n)$ на остальное.

7.3 Балансировка: средний и худший случай

- **Средний случай.** Куски выровнены по числу рёбер, нагрузка распределена ровно, ускорение близко к линейному до потолка ввода-вывода.
- **Худший случай (гипер-узел).** Одна вершина-приёмник степени D неделима, поэтому отставание ограничено величиной $\mathcal{O}(D)$. При разбиении на $K \gg P$ кусков и $D \ll m/K$ это отставание незаметно. Патология наступила бы, только если D сравнима с m/P , то есть одна вершина собирает заметную долю всех рёбер.

7.4 Сходимость

Степенной метод на расширенном графе сходится к единственному пределу. Двусторонний фоновый узел делает цепь Маркова сильно связной и апериодичной, отсюда (теорема Перрона–Фробениуса) существование и единственность стационарного распределения. Заодно это решает проблему висячих вершин: их масса уходит фоновому узлу и потом перераспределяется. Ошибка убывает геометрически: $\|s^{(t)} - s^*\|_1 \leq C \rho^t$, где $\rho < 1$ определяется вторым по модулю собственным значением матрицы перехода. Число итераций $T = \mathcal{O}(\log(1/\varepsilon)/\log(1/\rho))$.

7.5 Точность и воспроизводимость

Все суммы (и в ячейках (2), и в редукциях) считаются в фиксированном порядке, заданном раскладкой файла `sources` и порядком потоков. Сложение `double` не ассоциативно, поэтому именно фиксированный порядок даёт побитовую воспроизводимость и независимость от числа потоков. Накопление ошибки округления контролируется редкой перенормировкой к сохраняемой сумме масс n . Про конкретный критерий остановки и наблюдаемое число итераций на реальных графах — в разделе 8.

8 Тестирование и результаты

Тесты решают две задачи: подтвердить корректность и показать работающий результат под лимитом памяти. Стек — Gradle, JUnit 5, AssertJ. Всего 106 тестов, все зелёные.

8.1 Синтетические тесты (кратко)

- **Эталон.** Главный инструмент проверки — небольшая заведомо верная плотная реализация LeaderRank: строится матрица перехода $(n+1) \times (n+1)$ и считается степенной метод. Сложность $\mathcal{O}(n^2)$ ограничивает её малыми графами, но на них она даёт «истину», с которой сверяется основной движок. На примере `small.csv` расхождение $3,3 \cdot 10^{-15}$, то есть на уровне ошибки округления.
- **Графы вручную.** Треугольник, звезда (это и есть гипер-узел, центр обязан получить наибольший ранг), цепочка, висячая вершина, петля, две несвязные компоненты. Каждый случай фиксирует своё свойство.
- **Инварианты.** Сохранение массы ($\sum s \approx n$), положительность рангов, сходимость за ограниченное число итераций.
- **Главный тест — инвариантность к числу потоков.** Запуск с $P = 1$ и $P = 8$ даёт байт в байт одинаковый результат. Один этот тест сразу доказывает и корректность параллелизма (нет гонок), и детерминизм (нет зависимости от порядка).
- **Генератор данных.** Свой детерминированный генератор R-MAT со степенным распределением — чтобы в графе реально были гипер-узлы. Зерно фиксировано, граф повторяется.
- **Тест под лимитом памяти.** Полный конвейер запускается как отдельный процесс под лимитом `sgroup`. Тест проверяет завершение и пиковый RSS под

лимитом. Если на машине нет `cgroup` или `systemd-run` не может выставить лимит, тест автоматически пропускается, а не падает.

8.2 Реальные графы

Замеры сделаны скриптом `scripts/benchmark.sh`. Стенд: 24 ядра, ≈ 29 ГБ RAM, SSD, OpenJDK 21, Serial GC. Движок параллельный out-of-core, бюджет памяти берётся из `-Xmx`. Время `compute` — «горячее» (последний прогон при `--repeat=5`). Время `preprocess` однократное, включает разбор CSV и внешнюю сортировку.

Масштабирование по потокам. Граф R-MAT (scale=20), 16 млн рёбер, $n = 640\,324$. Число итераций — 38 при любом P , результат бит в бит не зависит от P .

P	preprocess, мс	compute, мс	ускорение	эффективность	пик RSS, МиБ
1	3450,9	1709,7	1,00	100%	260,6
2	1911,8	915,5	1,87	93%	220,0
4	1794,1	543,7	3,14	79%	218,6
8	1754,6	371,0	4,61	58%	219,7
16	1758,8	269,2	6,35	40%	222,0
24	1772,3	231,4	7,39	31%	245,6

Счётное ядро — это потоковый SpMV, упирающийся в пропускную способность памяти. Поэтому эффективность падает с ростом P : полоса памяти насыщается около $P = 8$ –16. Это честное свойство задачи, а не баг. Препроцессинг ускоряется примерно вдвое и выходит на плато: остаток — это последовательная сборка id-карты и сортировка. Пик RSS почти не зависит от P .

Реальные графы SNAP. $P = 24$, `-Xmx2g`, лимит итераций 100.

граф	n	m	preprocess, мс	compute, мс	пик RSS, МиБ
web-Google	875 713	5 105 039	703,9	405,3	237,9
soc-LiveJournal1	4 847 571	68 993 773	8174,7	3327,9	1235,1

Оба графа считаются out-of-core: рёбра не держатся в памяти. soc-LiveJournal1 с 69 млн рёбер обрабатывается под кучей 2 ГБ.

Сходимость на реальных графах. Критерий остановки масштабируется числом вершин: $\sum_i |s_{\text{new}}[i] - s_{\text{old}}[i]| < \text{tol} \cdot n$, $\text{tol} = 10^{-8}$. Это важно: масса сохраняется и равна n , поэтому ненормированная разность растёт с n . Фиксированный абсолютный порог на большом графе требовал бы изменения меньше уровня шума округления и никогда бы не срабатывал. Деление на n делает «сошлось» одинаковым на любом масштабе. Истинное число итераций до сходимости (замерено поднятием лимита):

граф	итераций до сходимости
R-MAT (scale=20)	38
web-Google	191
soc-LiveJournal1	1909

Реальные графы, особенно социальная сеть, имеют меньший спектральный зазор, поэтому степенной метод сходится медленнее. Оценка «десятки итераций» верна для графов вроде R-MAT, но не для соцсети. При этом ранжирование лидеров стабилизируется намного раньше численной сходимости: top-5 лидеров совпадает (те же вершины и тот же порядок) при лимите 100 и при полной сходимости для обоих графов. Медленный «хвост» лишь уточняет младшие разряды очков, не меняя порядок лидеров. Значит лимит 100 даёт правильных лидеров даже на LiveJournal, а флаг `--max-iterations` позволяет при желании довести счёт до полной сходимости.

RSS в зависимости от `-Xmx`. На том же R-MAT при сжатии кучи с 2 ГБ до 128 МБ время `compute` остаётся постоянным (≈ 215 – 226 мс), а RSS держится в районе 190–305 МиБ. Движок не деградирует под давлением памяти, а RSS не следит за размером графа (рёбра на диске). Главный рычаг RSS — это `-Xmx`. Нижняя граница памяти: web-Google считается до `-Xmx48m`, soc-LiveJournal1 до `-Xmx224m`. На границе всё упирается уже в честный $\mathcal{O}(n)$ (векторы значений фазы счёта), а не в предобработку — ровно тот предел задачи, обозначенный в разделе 6.5.

9 Заключение

Спроектировано и реализовано вычисление LeaderRank на ориентированном невзвешенном графе, который не помещается в память. Ключевые решения:

- **Метрика — LeaderRank.** Её ядро — один потоковый проход по рёбрам ($A^T \cdot \text{contrib}$), что и делает задачу разрешимой во внешней памяти.
- **Хранение — CSC по приёмнику.** Указатели `sourcesPtr` ($\mathcal{O}(n)$) в ОЗУ, источники `sources` ($\mathcal{O}(m)$) на диске, чтение последовательное.
- **Параллелизм — модель pull с разбиением по приёмнику.** Без блокировок и детерминированно при любом числе потоков.
- **Память — ограничена числом вершин.** Лимит задаётся снаружи (`-Xmx`, `sgroup`), программа под него подстраивается, пик RSS измеряется и приводится.

Эти решения прямо отвечают критериям задачи: эффективность на больших графах, использование всех ядер, устойчивость к гипер-узлам, воспроизводимость результата и чистота кода.

Главные числа. Параллельное ядро даёт ускорение $7,39\times$ на 24 потоках, детерминированно (38 итераций при любом P). Реальные графы SNAP считаются out-of-core: web-Google (5,1 млн рёбер) — около 0,4 с, soc-LiveJournal1 (69 млн рёбер) — около 3,3 с. Контрольный наивный вариант под тем же лимитом памяти падает, а потоковый движок завершается — это и подтверждает ценность out-of-core подхода.